Gráficos por computador

OpenGL

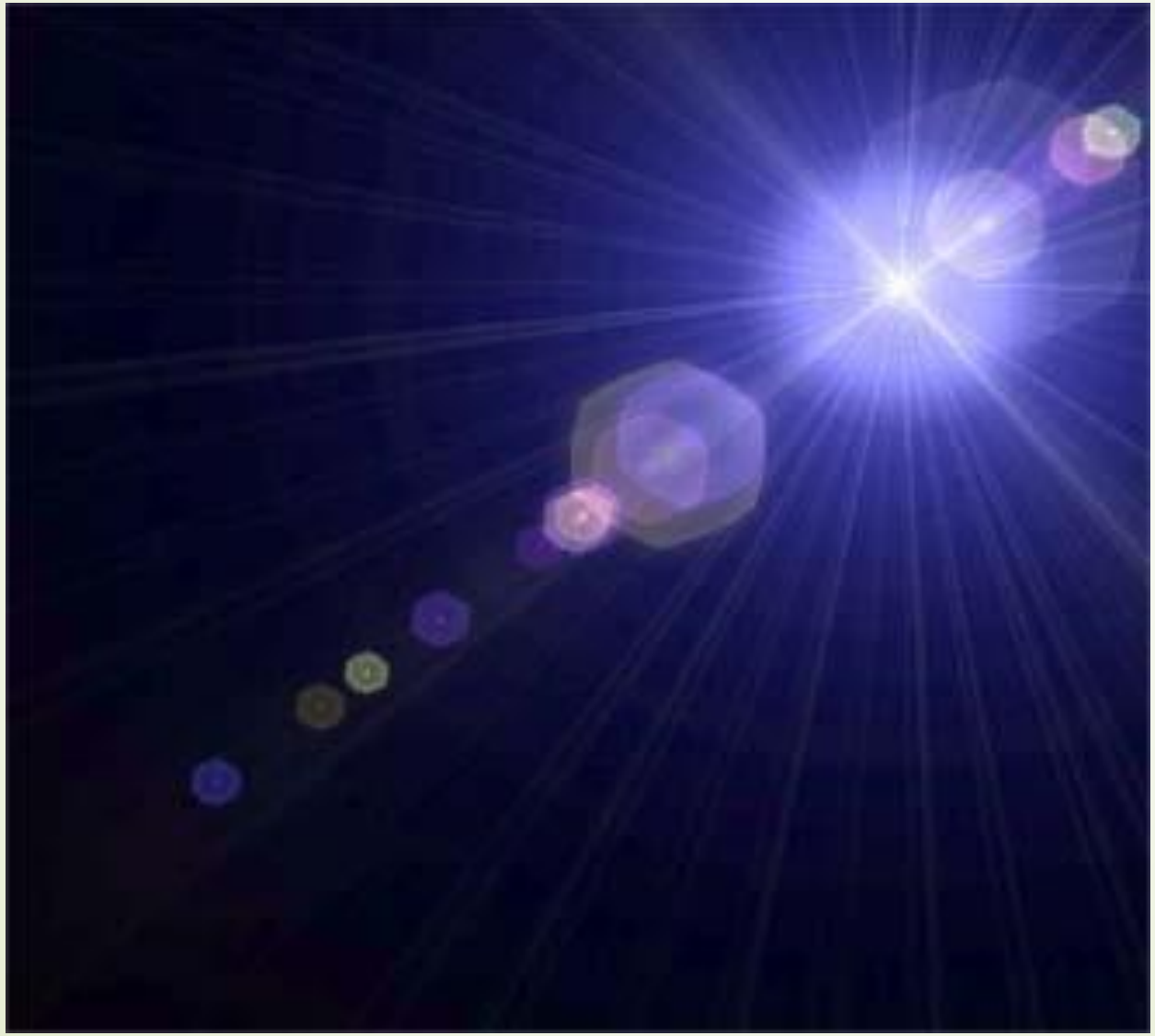
Introducción

Introducción

OpenGL es un estándar en gráficos por computador.

- Creada en 1992, ha ido evolucionando sus versiones por varias empresas punteras de hardware y software, como Silicon Graphics, Sun Microsystems, IBM y Hewlett Packard.
- Un hito importante fue su elección por parte de John Carmack para renderizar el juego Quake.
  - Hizo una comparación entre DirectX (Microsoft) y OpenGL con un ejemplo de Lens Flare (brillo sobre lentes).
  - Resolvió el ejemplo con 30 líneas de código frente a 300 líneas de código en DirectX.
  - El efecto era más rápido y mejor con OpenGL.
- La última versión principal es OpenGL 4.6. Khronos continúa manteniendo la especificación, aunque el desarrollo de nuevas APIs gráficas de bajo nivel se centra principalmente en Vulkan

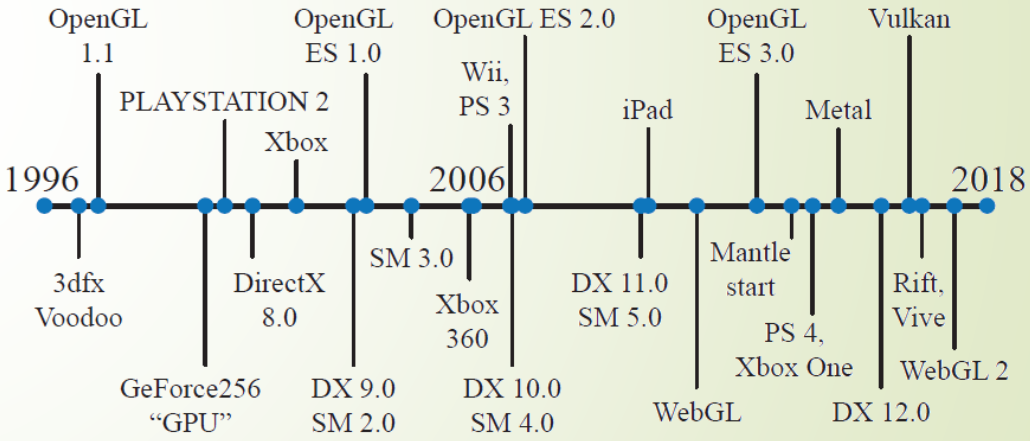
Introducción



Introducción

En las primeras versiones, el renderizado de gráficos solo se podía realizar utilizando funciones fijas de la librería, lo que dificultaba mucho la personalización.

- A partir de la versión 2.0 se introduce la programación flexible de la GPU utilizando shaders mediante el lenguaje GLSL.
- A partir de la versión 3.1 se elimina la mayor parte del pipeline fijo del perfil core.
- La última especificación de OpenGL es la 4.6, que continúa siendo una muy buena opción para el aprendizaje de programación gráfica, ya que sienta las bases para desarrollos más recientes como Vulkan.
- Existe una variante, OpenGL ES (OpenGL for Embedded Systems), que es una API simplificada diseñada para dispositivos integrados como móviles y videoconsolas (modificaciones utilizadas en Playstation y Wii).



Requisitos de nuestra práctica

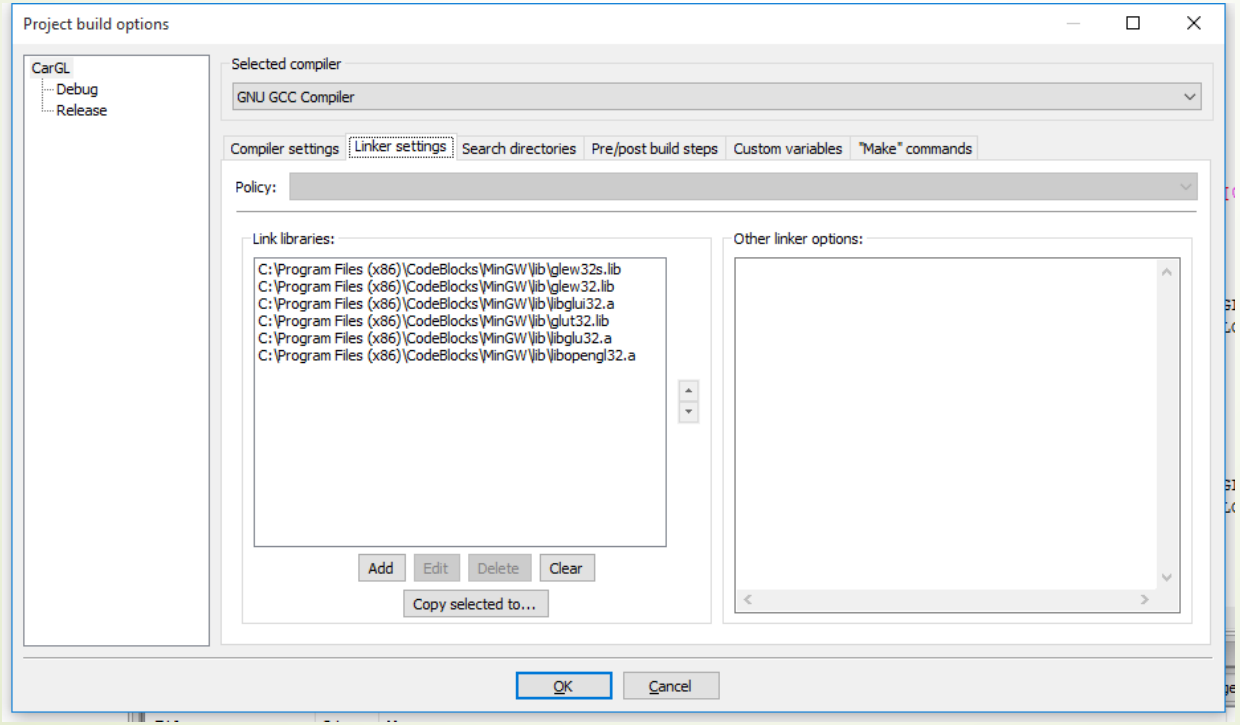
OpenGL es compatible con Windows, Unix, Linux, Mac OS…

- Se requiere de un compilador de C++ para generar los ejecutables.
- **Librería principal y librerías adicionales**:
  - **Librería básica: gl**
  - Librería con utilidades básicas: glu
  - Librería para cargar y acceder a funciones y extensiones de OpenGL: glew
  - Librería matemática para vectores, matrices y transformaciones: glm
  - Librería con utilidades de interfaz: glui y glut
    - Glut permite creación de ventanas, gestión de eventos de entrada y menús multiplataforma.
- Para utilizar OpenGL con CodeBlocks necesitamos:
  - Instalar el compilador completo MinGW (las librerías de OpenGL están incluidas).
  - Para utilizar la librería glut:
    - glut.h, glut.lib, glut32.lib, glut.dll, glut32.dll

Configuración

Project → Build Options → Linker Settings:

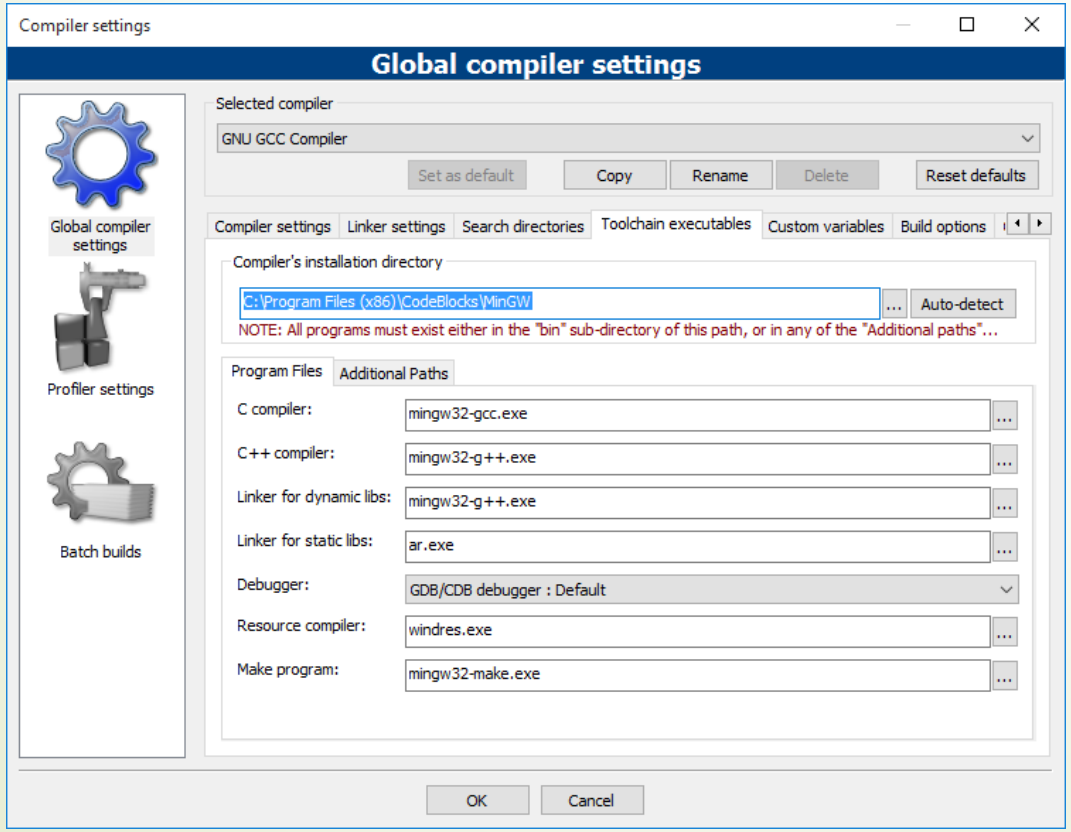
- Se debe añadir las librerías en este orden (el orden importa).
- Se debe copiar todo el contenido de la carpeta “Para Copiar” como se explica al principio del archivo main.cpp.



Configuración

Settings → Compiler → Toolchain executables:

- Se debe tener la ruta del compilador MinGW apuntando al que hay en el interior de la carpeta Codeblocks.



Funcionamiento

OpenGL es una API estandarizada para comunicarse con la GPU.

- El programa crea y configura recursos gráficos (buffers, texturas, shaders…) y se mantiene ese estado accesible para la GPU.
- Cuando se dibuja, la API interpreta los datos guardados sin tener que ser reenviados cada vez.
- Funciona como una máquina de estados.
  - Se especifica un estado y las operaciones posteriores utilizan el estado actualmente configurado hasta que se modifica.
  - Existe una enorme lista de comandos que activan diferentes capacidades de la librería, modificando su estado.
    - Por ejemplo, los comandos glEnable y glDisable acompañados de diferentes flags permiten activar diferentes funcionalidades como el culling o los tests de profundidad del buffer de profundidad.
- Guía de referencia:
  - https://www.khronos.org/files/opengl46-quick-reference-card.pdf

Sintaxis

La librería tiene una sintaxis específica para distinguir a sus funciones, constantes y tipos de parámetros que acepta una función:

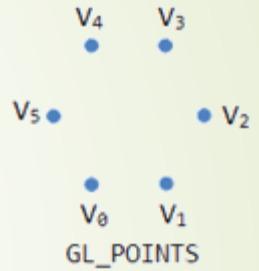
- Las funciones suelen tener como prefijo **gl**
- Las constantes suelen tener prefijo **GL_**
- Los sufijos de algunas funciones indican el número y tipo de parámetros.

Tipos de datos

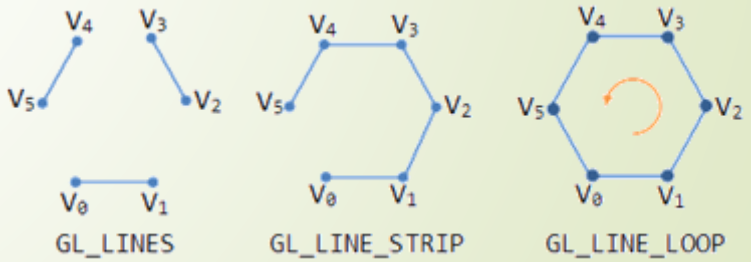
Sufijo	Tipos de Datos	Definición en C	Definición en OpenGL
b	Entero 8 bits	signed char	GLbyte
s i	Entero 16 bits Entero 32 bits	short int	GLshort GLint, GLsizei
f	Real 32 bits	float	GLfloat, GLclampf
d	Real 64 bits	double	GLdouble
ub	Entero positivo 8 bits	unsigned char	GLubyte. GLboolean
us	Entero positivo 16 bits	unsigned short	GLushort
ui	Entero positivo 32 bits	unsigned int	GLuint, GLenum, GLbitfield

Primitivas

- **Puntos**
  - Se definen por un único vértice.
  - **GL_POINTS**
    - El tamaño indica la cantidad de píxeles que ocupa cada lado del cuadrado definido por el punto.

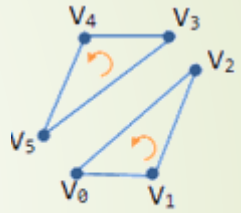


- **Líneas**
  - Se definen por pares de vértices.
  - **GL_LINES**
    - Vértices 0 y 1 forman una línea, vértices 2 y 3 forman la siguiente línea…
  - **GL_LINE_STRIP**
    - Los vértices adyacentes forman líneas.
  - **GL_LINE_LOOP**
    - Igual que line_strip, excepto que el último y el primer vértice forman otra línea.

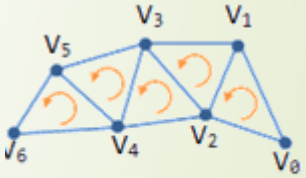


Primitivas

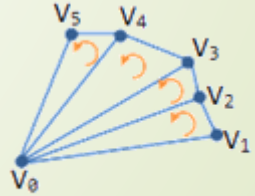
- **Triángulos**
- Se definen por 3 vértices.
  - **GL_TRIANGLES**
    - Vértices 0, 1 y 2 forman un triángulo. Vértices 3, 4 y 5 el siguiente…
  - **GL_TRIANGLE_STRIP**
    - Cada grupo de 3 vértices adyacentes forma un triángulo.
    - La dirección a la que apunta la cara del triángulo viene determinada por la del primer triángulo.
    - Cada triángulo sucesivo tiene el orden de sus caras al revés, por lo que el sistema lo compensa.
  - **GL_TRIANGLE_FAN**
    - El primer vértice permanece fijo.
    - Cada grupo de 2 vértices consecutivos forma un triángulo con el primero.



GL_TRIANGLES



GL_TRIANGLE_STRIP



GL_TRIANGLE_FAN

Primitivas

Estas primitivas gráficas se envían a los shaders para ser dibujadas (anteriormente, se han tenido que definir los atributos de cada vértice).

- **Función:**
  - *void glDrawArrays(GLenum mode, GLint first, GLsizei count);*
- **Parámetros:**
  - *mode*:
    - Especifica el tipo de primitiva a dibujar mediante las constantes simbólicas explicadas.
  - *first*:
    - Especifica el índice desde el comienzo del vector (array) habilitado.
  - *count*:
    - Especifica el número de vértices que se procesan a partir de first.

Shaders

Los shaders son funciones en lenguaje GLSL que se ejecutan en la GPU y que permiten realizar cálculos necesarios para la renderización de las primitivas.

Existen dos funciones muy importantes y que deberemos programar obligatoriamente:

- **Vertex shader.**
  - Se ejecuta una vez por vértice de la primitiva.
  - Encargado de generar la posición final del vértice transformado en coordenadas de recorte (multiplicando por la matriz **M**odel**V**iew**P**rojection).
  - La posición final se almacena en la variable de salida **gl_Position**.
    - Se trata de una palabra clave solo disponible en el Vertex shader.



Pipeline resumido de OpenGL

Shaders

- **Fragment shader.**
  - Se ejecuta una vez por cada fragmento generado durante la rasterización de las primitivas.
  - Cada fragmento contiene la información necesaria para generar un posible valor del framebuffer, como color, profundidad y valores interpolados.
  - Encargado de generar el color final de cada fragmento de la primitiva.
  - El color final se almacena en la variable de salida **gl_FragColor**.
    - Se trata de una palabra clave solo disponible en el Fragment shader.
    - Realmente, en las últimas versiones esta variable es eliminada y se requiere definir variables de salida con out, pero en contextos de compatibilidad se sigue usando.



Pipeline resumido de OpenGL

Shaders

Ejemplo de Vertex shader básico:

```
attribute vec4 a_Position; //In: Posición del vértice
attribute vec4 a_Color; //In: Color del vértice
varying vec4 v_Color; //Out: color para frag shader
void main()
{
     gl_Position = a_Position; //Pos del vértice
     v_Color = a_Color; //Color del vértice
}
```

Código Fuente GLSL

Shaders

Ejemplo de Fragment shader básico:

```
varying vec4 v_Color; //In: color interpolado
void main()
{
      gl_FragColor = v_Color;
}
```

Código Fuente GLSL